Text Classifier Using K-Nearest Neighbors(k-NN)

QUAN DO - SCU ID: 07700006209
(qldo@scu.edu)

Santa Clara University, CA, 95053

The report contains a customized k-nearest neighbors (k-NN) text classification model built using core Python libraries. The system works on documents through tokenization, builds a sparse document-term matrix, TF-IDF weights, and classifies texts based on cosine similarity as well as majority voting with a tie-breaking mechanism. With accuracy of 95 percent, the report outlines the technical process, accompanying mathematics, and recommends enhancements for better performance.

0 Introduction

The report below demonstrates the step-by-step implementation of a text classification pipeline using a custom k-Nearest Neighbors (k-NN) model to classify documents of a corpus into pre-defined categories by the similarity of content. The process begins with raw text preprocessing, sparse matrix construction, TF-IDF transformation, and then classification by cosine similarity. The K-NN model is built using foundational python libraries without relying on high-level machine learning libraries like scikit-learn, thus having full control over each aspect of the pipeline.

1 Libraries and Environment Setup

The code utilizes several basic Python libraries. *numpy* is utilized for numerical array operations, while pandas handles structured data. *scipy.sparse* enables the utilization of memory-efficient sparse matrix representations, i.e., CSR (Compressed Sparse Row) format, which is highly efficient in memory saving for high-dimensional data such as text. *matplotlib.pyplot* is utilized for visualization, specifically plotting the confusion matrix. The *collections* module provides the *Counter* and *defaultdict* data structures for count and aggregation operations within classification tasks. The library *NLTK* facilitates natural language processing operations like stemming (*PorterStemmer*), lemmatization (*WordNetLemmatizer*), and stopword removal (*stopwords.words('english')*).

2 Data Preprocessing Steps

Data preprocessing is a critical step in the text classification process because it determines the representational quality of the input features used for similarity-based learning. In this case, the core preprocessing logic is encapsulated in the *clean_text* function, which is responsible for transforming each raw document into a cleaned list of tokens suitable for matrix encoding.

2.0.1 Training Set Data Visualization

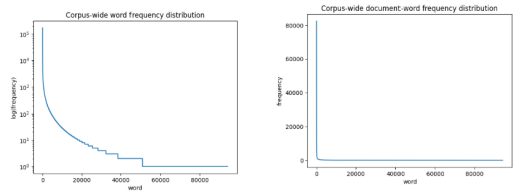

Fig. 1: Data Visualization Of Training Data.

The two plots highlight the corpus's highly skewed word distributions. The first shows a pattern where a few words dominate in frequency while most are rare. The second confirms that many words appear in only a few documents, indicating high sparsity. These patterns suggest the importance of preprocessing steps like stopword removal and TF-IDF weighting to reduce noise and emphasize meaningful terms. These steps are effective in reducing the dimensionality of the sparse matrix

2.1 Data Cleaning and Tokenization

In this section, I defined a text preprocessing function, *clean_text*, to prepare raw textual data for as a step prior to constructing a sparse matrix. To enhance runtime efficiency, I precompiled regular expressions for removing numeric characters, special symbols, and short words (defined as words with two or fewer characters). Additionally, I converted the list of English stop words from NLTK into a Python set to enable faster membership checking during token filtering. I also initialized a stemmer(PorterStemmer) to apply stemming for word normalization. Initially, lem-

matizer (WordNetLemmatizer) was chosen instead of stemmer; however, stemmer was able to produce slightly better accuracy. One notable difference between these 2 methods are the outcome. After using stemmer, *64615* features were recorded; on the other hand, that value was *77320* for lemmatizer. Therefore, this slight increase in terms of accuracy between the two methods might come from the decrease in the dimensionality of the data.

The *clean_text* function processes each document by first converting it to a string, then applying the compiled regular expressions sequentially to clean the text. Following this, each document is tokenized using whitespace splitting. Tokens are then filtered in order to exclude stop words and then passed through the stemming process. The final output is a list of token lists, where each sublist corresponds to the cleaned and stemmed tokens of a single document. This structured and normalized token output is suitable for critical tasks such as sparse matrix construction.

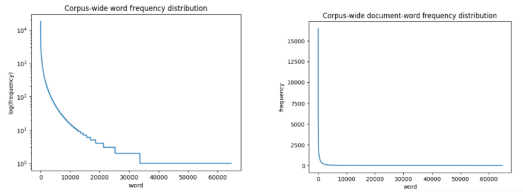

Fig. 2: Data Visualization Of Cleaned Training Data.

After the cleaning process was proceeded (including features reduction steps) the distribution of the text remains unchanged; thus, ensuring that this is still a representative sample of the whole training data.

3 Build, Scale, and Normalize Sparse Matrix

This process transforms a collection of tokenized and cleaned documents into a compressed sparse matrix using word frequencies. It then applies TF-IDF scaling to emphasize important but rare terms across the documents. Finally, L2 normalization is used to ensure that all document vectors have unit length, which is essential to reduce time complexity for cosine similarity computation. This prepares the data for efficient and meaningful comparisons in models like k-NN.

3.1 Construct Compressed Sparse Matrix

The *build_matrix* function constructs a sparse document-term matrix in Compressed Sparse Row (CSR) format from a list of tokenized documents. Each document is a list of terms, and the function maps each unique term to a unique index (stored in idx). It first counts the number of non-zero entries (distinct words per document), then allocates memory for the sparse matrix components: ind for column indices, val for term frequencies, and ptr for row pointers.

As it loops through each document, it creates a Counter of term frequencies and filters out words not in the index (if is_training=False). It fills the matrix arrays accordingly. Finally, it assembles and returns the sparse matrix—along with the index mapping if training. There has been a few modifications for this function as compared to the function presented in *activity 3* in order to ensure that the test CSR matrix would have the same number of features as the train CSR matrix. This modification comes from the part where we add the condition where we only add unique words into the idx dictionary when we are working with the training data set; in other words, the test data set is only evaluated based on the tokens that they have in common with the training data set.

3.2 Scale Using TF-IDF

$$\text{IDF}(t) = \log\left(\frac{N}{1+\text{DF}(t)}\right) \tag{1}$$

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t) \tag{2}$$

This section applies Term Frequency-Inverse Document Frequency (TF-IDF)(2) scaling to a sparse document-term matrix in Compressed Sparse Row (CSR) format. TF-IDF is a measure used to determine the relevance of a word to a document in a collection. The function first computes the Term Frequency (TF) by normalizing the term counts of each document by its length. Next, it determines every term's Document Frequency (DF) (how many documents include the term—by reading the matrix and keeping track of unique words on a per-row basis). Then it computes the Inverse Document Frequency (IDF) based on the formula (1), where N is the number of documents. Finally, it multiplies each non-zero element in the matrix by the IDF of the respective term, calculating the TF-IDF score in effect. The function returns the scaled matrix which would be further modified as part of the text-classification process.

Applying TF-IDF would help in increasing the accuracy of the model because it helps reduce the influence of common words that have no significant contribution on the label of a document and helps resolving the issue in which we treat every word as being equally significant; instead, this method gives more weighting to rarer and more significant words. I also compared this method with IDF scailing (provided in activity 3) and noticed that there was a slightly higher accuracy; thus, I decided to proceed with this method.

3.3 Normalize using L2 Norm

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2} \tag{3}$$

After the CSR matrix was scaled using TF-IDF, I then normalize the matrix using formula (3). This process would help in optimizing the time spent for calculating the *Cosine Similarity* as it would simplify the whole computation steps to just being a dot product. Therefore, this would help in optimizing the time complexity of the overall classification process

4 Classification

This section will detail the algorithms underlying the *classify* function which form a fundamental part of the kNN model.

4.1 Proximity Function

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n \tag{4}$$

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i \tag{5}$$

The proximity function being used for this custom kNN model would be **Cosine Similarity**. However, within the function, we will only compute the dot product between input data and all of the data points from training data set. Another consideration for this proximity function would be euclidian distance; however, this distance metric is easily affected by the **Curse of Dimensionality**; thus won't be suitable for the task of text classification as there are a lot of features involved in this task.

4.2 Classify Function

This *classify* function will then use the *proximity*(cosine similarity) function defined above as the distance metric to implement the k-nearest neighbors algorithm for text classification. This function will receive a test document x and then it would calculate this document's similarity scores against all training vectors in the *training data*. Afterwards, it would sort the similarity scores in descending order and select the top k most similar documents and extract its *label*. The function will then carry out **weighted vote** by accumulating the total similarity score of each label and return the label with the highest accumulated score as the prediction.

4.3 Parameter Tuning

To find the best possible k-value, I have attempted to try the k value from 4 to 9 and after many trials, the k value with the highest recorded accuracy is 6.

5 Result

The k-NN text classifier model achieved an F1 score of **95.12 percent and achieving the third highest ranking** at this moment, demonstrating highly effective performance on the test set. This indicates the strength of the preprocessing approach utilizing the *clean_text* function, which normalized and filtered input text effectively. TF-IDF weighting preserved considerable term distinctions, while cosine similarity enabled proper comparison of document vectors. Utilizing weighted voting further increased classification reliability, with proper and consistent label predictions.

6 Future Modification

Future modification to this model is to incorporate n-gram features in order to capture local word context and semantic phrases, which are often overlooked when using unigrams. The reason why this current model does not integrate this feature is because it would significantly expand the dimensionality of the data; thus, can undermine the significance of the proximity measurment and lower the accuracy of the model. In the future, integrating dimensionality reduction techniques such as PCA or Truncated SVD (LSA) can help resolve the problems stated above.

THE AUTHORS